TD Clustering – Compte rendu

NGUYEN Dinh Huy NGUYEN Vinh Tri

Data Mining – INSA Centre Val de Loire – Novembre 2026

Table des matières

Organisation du rendu et utilisation du code **1**

1 Clustering k-means **2**
1.1 Exercice 1 : implémentation de k-means 2
1.2 Exercice 2 : tests sur le jeu de données [[2], [4], [6], [12], [24], [30]] 4
1.3 Exercice 3 : visualisations 2D 5
1.4 Exercice 4 : comparaison avec SciKit-Learn 6

2 Clustering hiérarchique **7**
2.1 Matrice de distance 7
2.2 Matrice de similarité 9

3 Étude de cas **10**
3.1 Exercice 1 : préparation des données 10
3.2 Exercice 2 : clustering et interprétation 11

Organisation du rendu et utilisation du code

Contenu de l'archive.

- `section1_kmeans.py` : notre implémentation de k-means (Section 1, Exercices 1 à 3) ;
- `section1_sklearn.py` : la comparaison avec SciKit-Learn (Section 1, Exercice 4) ;
- `section1_verifications.py` : le calcul des chiffres cités dans nos discussions de la Section 1 ;
- `section2_hierarchique.py` : le clustering hiérarchique (Section 2) ;
- `section3_etude_de_cas.py` : l'étude de cas (Section 3) ;
- `data/Country-data.csv` : le jeu de données de l'étude de cas ;
- `figures/` : les figures générées par les scripts (ce sont elles qui sont insérées dans ce compte rendu) ;
- `requirements.txt` : les bibliothèques et leurs versions ;
- `README.md` : un résumé de l'installation et de l'exécution ;
- `rapport/` : ce compte rendu (PDF) et son source LaTeX.

Installation. Notre distribution Linux refuse `pip install` en dehors d'un environnement virtuel. Nous avons donc installé NumPy, Matplotlib, SciPy et SciKit-Learn dans un environnement virtuel, avec les versions listées dans `requirements.txt` :

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

Exécution. Les commandes suivantes se lancent depuis la racine du projet. Les scripts trouvent eux-mêmes le fichier de données et le dossier `figures/`, quel que soit le dossier courant.

Commande	Résultat
`.venv/bin/python section1_kmeans.py`	Exercices 1 à 3, figures `ex3_*.png`
`.venv/bin/python section1_sklearn.py`	Exercice 4
`.venv/bin/python section1_verifications.py`	Vérifications de la Section 1
`.venv/bin/python section2_hierarchique.py`	Section 2, figures `s2_*.png`
`.venv/bin/python section3_etude_de_cas.py`	Section 3, figures `s3_*.png`

Vérification. Chaque script affiche de quoi comparer nos résultats avec ceux de l'énoncé :

- `section1_kmeans.py` affiche toutes les itérations de k-means, sous la même forme que l'énoncé ;
- `section1_verifications.py` compare automatiquement cet affichage avec celui de l'énoncé, et recalcule les chiffres de nos discussions (tirages aléatoires, recherche exhaustive de la meilleure partition) ;
- `section1_sklearn.py` indique, pour chaque configuration des Exercices 1 à 3, si SciKit-Learn trouve les mêmes clusters et la même SSE que notre code ;
- `section2_hierarchique.py` affiche les fusions successives (clusters fusionnés et distance), à comparer avec les dendrogrammes de l'énoncé ;
- `section3_etude_de_cas.py` affiche le début de `data_reduced` (à comparer avec celui de l'énoncé), la part de variance expliquée et la corrélation de chaque variable avec les deux axes ; pour k-means, la SSE, la taille des clusters et la moyenne de quelques variables dans chaque cluster, ainsi que les résultats de 200 tirages aléatoires des centroïdes initiaux ; pour le clustering hiérarchique, la taille des clusters obtenus en coupant chaque dendrogramme en 2 puis 3 clusters, et la comparaison de la coupe en 3 clusters du complete link avec k-means.

Les centroïdes initiaux tirés au hasard dépendent d'une graine fixée dans le code. Deux exécutions successives produisent donc exactement les mêmes résultats et les mêmes figures. La seule exception est l'exemple de l'Exercice 1 avec tirage aléatoire, qui illustre justement ce tirage.

1 Clustering k-means

1.1 Exercice 1 : implémentation de k-means

Notre implémentation se trouve dans `section1_kmeans.py`. L'algorithme n'utilise aucune bibliothèque externe, seulement les modules standard `math` et `random` ; Matplotlib ne sert qu'aux figures de l'Exercice 3.

Représentation des données. Comme dans l'énoncé, un point est une liste de nombres (`[1]` en 1D, `[-2, 3]` en 2D) et un jeu de données est une liste de points. Aucune de nos fonctions ne suppose une dimension particulière : les calculs se font coordonnée par coordonnée, donc le même code fonctionne en 1D, en 2D ou plus. Si deux points n'ont pas la même dimension, le calcul de distance lève une erreur.

Paramètres. La fonction prend les trois paramètres demandés :

```
kmeans(data, k, centroides_initiaux=None)
```

Elle affiche les résultats intermédiaires et retourne la SSE. Les paramètres optionnels ajoutés ensuite sont présentés à l'Exercice 3.

Déroulement de l'algorithme.

1. **Initialisation.** Si `centroides_initiaux` vaut `None`, `random.sample(data, k)` tire k points du jeu de données, sans remise. Nous prenons des points du jeu plutôt que des coordonnées quelconques : chaque centroïde initial a ainsi au moins un point proche (lui-même), ce qui évite un cluster vide à la première itération lorsque les points du jeu sont distincts. Si le jeu contient des points en double, deux centroïdes peuvent coïncider ; c'est alors la règle du cluster vide (étape 3) qui s'applique.
2. **Affectation.** Chaque point est affecté au centroïde le plus proche pour la distance euclidienne
$$d(p, q) = \sqrt{\sum_i (p_i - q_i)^2}.$$
En cas d'égalité, le point va au centroïde de plus petit indice. Ce cas se présente à l'Exercice 2.
3. **Mise à jour.** Le nouveau centroïde d'un cluster est la moyenne de ses points, coordonnée par coordonnée. Si un cluster est vide, il garde son ancien centroïde, sinon la moyenne serait une division par zéro.
4. **Arrêt.** On répète les étapes 2 et 3 jusqu'à ce que l'affectation des points soit identique à celle de l'itération précédente. À ce moment, les centroïdes recalculés seraient les mêmes, donc plus rien ne peut changer. C'est aussi ce que montre l'énoncé : la dernière itération affichée répète l'affectation de la précédente.
5. **Erreur.** La SSE est la somme, sur tous les clusters, des carrés des distances entre chaque point et le centroïde de son cluster :
$$\mathrm{SSE} = \sum_{j=1}^{k} \sum_{x \in C_j} d(x, c_j)^2.$$
Nous additionnons directement $\sum_i (x_i - c_i)^2$, sans calculer la racine puis la remettre au carré, ce qui évite des erreurs d'arrondi inutiles.

Affichage. Chaque itération est affichée avec `print("Centroid", c, " Points", cluster)`, puis la SSE avec `print("Fin clustering, erreur = ", sse)`, ce qui donne les mêmes espaces que l'énoncé. À l'itération 1, les centroïdes sont affichés tels qu'ils ont été donnés (`[1]`) ; à partir de l'itération 2, ce sont des moyennes, donc des flottants (`[1.0]`), exactement comme dans l'énoncé.

Vérification. `section1_kmeans.py` lance les deux exemples de l'énoncé. Le script `section1_verifications.py` relance ces deux exemples, récupère l'affichage de notre fonction et le compare au texte de l'énoncé, recopié dans le script : les deux affichages sont identiques (`True` pour les deux configurations), avec une SSE de 98.0 et de 2.5.

Centroïdes initiaux tirés au hasard. Nous avons lancé k-means sur le même jeu de données avec 200 graines différentes : la SSE vaut 2.5 dans 140 cas (70 %) et 98.0 dans 60 cas (30 %). On peut l'expliquer en testant les $\binom{5}{3} = 10$ choix possibles de centroïdes initiaux : les 3 choix qui contiennent à la fois `[1]` et `[2]` donnent 98.0, les 7 autres donnent 2.5, ce qui correspond aux proportions observées. Dans ces 3 cas, `[1]` et `[2]` gardent chacun leur propre cluster, donc 18, 20 et 31 sont obligés de partager le troisième : l'algorithme converge, mais vers un minimum local. Le résultat de k-means dépend donc des centroïdes initiaux.

1.2 Exercice 2 : tests sur le jeu de données [[2], [4], [6], [12], [24], [30]]

Les deux configurations sont lancées par `section1_kmeans.py`. Voici l'affichage obtenu.

(a) Centroïdes initiaux `[[2], [6]]`

```
Iteration 1
Centroid [2]  Points [[2], [4]]
Centroid [6]  Points [[6], [12], [24], [30]]
Iteration 2
Centroid [3.0]  Points [[2], [4], [6]]
Centroid [18.0]  Points [[12], [24], [30]]
Iteration 3
Centroid [4.0]  Points [[2], [4], [6], [12]]
Centroid [22.0]  Points [[24], [30]]
Iteration 4
Centroid [6.0]  Points [[2], [4], [6], [12]]
Centroid [27.0]  Points [[24], [30]]
Fin clustering, erreur =  74.0
```

(b) Centroïdes initiaux `[[12], [24]]`

```
Iteration 1
Centroid [12]  Points [[2], [4], [6], [12]]
Centroid [24]  Points [[24], [30]]
Iteration 2
Centroid [6.0]  Points [[2], [4], [6], [12]]
Centroid [27.0]  Points [[24], [30]]
Fin clustering, erreur =  74.0
```

Configuration	Clusters finaux	Itérations	SSE
(a) `[[2], [6]]`	$\{2,4,6,12\}$, $\{24,30\}$	4	74.0
(b) `[[12], [24]]`	$\{2,4,6,12\}$, $\{24,30\}$	2	74.0

Discussion. Les deux configurations donnent exactement le même clustering, avec la même SSE de 74.0 : aucun des deux n'est meilleur que l'autre. La seule différence est la vitesse de convergence.

- Avec (b), le centroïde 12 est déjà dans le groupe de gauche et 24 dans celui de droite : la frontière entre eux (18) sépare déjà $\{2,4,6,12\}$ de $\{24,30\}$. L'affectation de l'itération 1 est donc la bonne, et l'itération 2 ne fait que le confirmer.
- Avec (a), les deux centroïdes initiaux sont dans le groupe de gauche. Le second centroïde doit se déplacer vers la droite au fil des itérations (6, puis 18, 22 et 27), et les points 6 et 12 changent de cluster en cours de route. Il faut donc 4 itérations.

Ce clustering est aussi le meilleur possible pour $k = 2$. En 1D, dans une partition optimale, chaque point est dans le cluster du centroïde le plus proche (sinon le déplacer ferait baisser la SSE), donc chaque cluster est un intervalle. Il suffit alors de tester les 5 façons de couper la liste triée en deux, ce que fait `section1_verifications.py` :

Coupure	SSE
$\{2\} \mid \{4,6,12,24,30\}$	516.8
$\{2,4\} \mid \{6,12,24,30\}$	362.0
$\{2,4,6\} \mid \{12,24,30\}$	176.0
$\{2,4,6,12\} \mid \{24,30\}$	**74.0**
$\{2,4,6,12,24\} \mid \{30\}$	315.2

Contrairement à l'Exercice 1, où l'initialisation `[[1], [2], [18]]` restait bloquée dans un minimum local, les deux initialisations atteignent ici l'optimum global.

Remarque sur l'égalité en (a). À l'itération 1, le point 4 est à la même distance (2) des centroïdes 2 et 6. Notre règle le met dans le premier cluster. Nous avons vérifié avec `section1_verifications.py` que l'autre choix (le mettre avec 6) mène aussi à une SSE finale de 74.0 : ici, le résultat ne dépend pas de la règle d'égalité.

1.3 Exercice 3 : visualisations 2D

Paramètres optionnels ajoutés. Nous avons ajouté trois paramètres optionnels à la fonction de l'Exercice 1 :

```
kmeans(data, k, centroides_initiaux=None, noms=None,
       prefixe_figures=None, retourner_clusters=False)
```

- `noms` : la liste des noms des points, dans le même ordre que `data` (par exemple `["M1", ..., "M6"]`). Chaque point est annoté avec son nom sur les figures. Avec `None`, les points ne sont pas annotés.
- `prefixe_figures` : si ce paramètre n'est pas `None`, la figure de l'itération i est sauvegardée dans le fichier `<prefixe_figures>_iter<i>.png` (le dossier est créé s'il n'existe pas). Nous sauvegardons les figures dans des fichiers plutôt que de les afficher, parce qu'elles sont insérées dans ce compte rendu et que le script peut ainsi tourner sans écran. Les figures n'ont de sens qu'en 2D : si un point n'a pas 2 coordonnées, la fonction lève une erreur.
- `retourner_clusters` : si ce paramètre vaut `True`, la fonction retourne le couple (SSE, clusters finaux) au lieu de la SSE seule. Il ne sert pas aux figures de cet exercice, mais à `section1_verifications.py` et à l'étude de cas (Section 3), qui ont besoin des clusters.

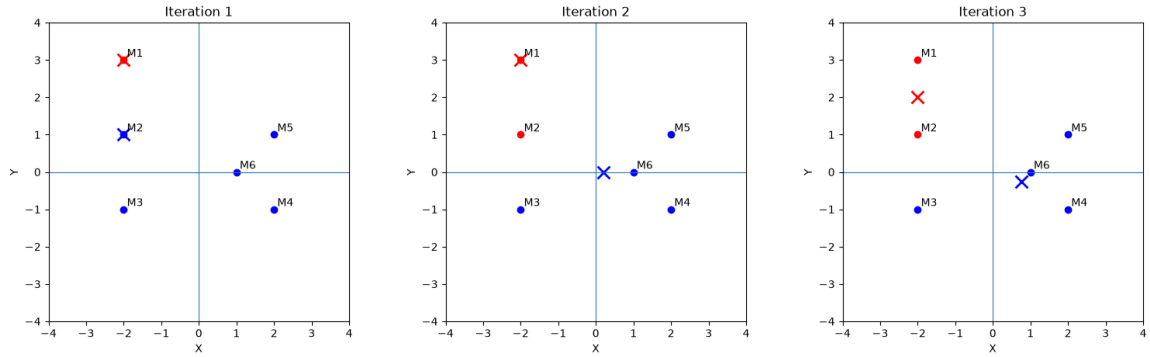
Avec les valeurs par défaut, la fonction se comporte exactement comme à l'Exercice 1.

Contenu des figures. À chaque itération, juste après l'affectation des points, la fonction dessine une figure qui correspond à ce qui est affiché pour cette itération :

- les points sont représentés par des $\bullet$ annotés avec leur nom, les centroïdes par des $\times$;
- le cluster j et son centroïde ont la même couleur (rouge pour le premier, bleu pour le second, comme dans l'énoncé). La couleur d'un point est calculée avec la même règle du centroïde le plus proche que l'affectation, donc la figure ne peut pas contredire l'affichage texte ;
- les axes $X = 0$ et $Y = 0$ sont tracés, et le cadre est symétrique autour de l'origine. La seule différence avec l'énoncé est une marge d'une unité (cadre $[-4, 4]$ au lieu de $[-3, 3]$), pour que M1 $(-2, 3)$ ne soit pas collé au bord.

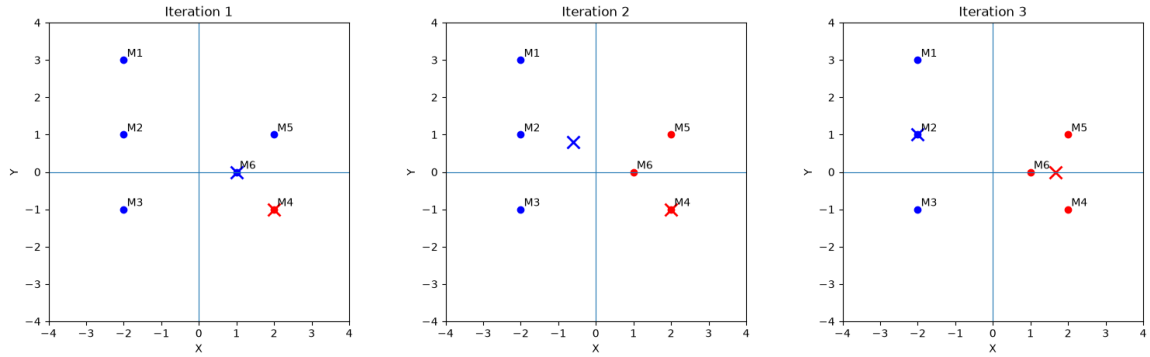
Reproduire les figures. La commande `.venv/bin/python section1_kmeans.py` génère les deux séquences dans `figures/`. Elles viennent de ces deux appels (extrait simplifié du script) :

```
M = [[-2, 3], [-2, 1], [-2, -1], [2, -1], [2, 1], [1, 0]]
noms_M = ["M1", "M2", "M3", "M4", "M5", "M6"]
kmeans(M, 2, [M[0], M[1]], noms=noms_M,
       prefixe_figures="figures/ex3_init_M1_M2")   # M1 et M2
kmeans(M, 2, [M[3], M[5]], noms=noms_M,
       prefixe_figures="figures/ex3_init_M4_M6")   # M4 et M6
```



(a) Itération 1 (b) Itération 2 (c) Itération 3

FIGURE 1 – Centroïdes initiaux M1 et M2 (fichiers `figures/ex3_init_M1_M2_iter<i>.png`). SSE finale : 15.5.



(a) Itération 1 (b) Itération 2 (c) Itération 3

FIGURE 2 – Centroïdes initiaux M4 et M6 (fichiers `figures/ex3_init_M4_M6_iter<i>.png`). SSE finale : 10.67.

Les deux séquences sont les mêmes que celles de l'énoncé : mêmes positions des centroïdes et mêmes couleurs à chaque itération.

1.4 Exercice 4 : comparaison avec SciKit-Learn

Le script `section1_sklearn.py` lance d'abord le code de l'énoncé, qui affiche bien `[0 1 2 2 2]` et `[0 0 1 1 2]`, puis compare notre fonction et SciKit-Learn sur les six configurations des Exercices 1 à 3.

Pourquoi cet affichage correspond aux clusters de l'Exercice 1. `labels_` donne, pour chaque point de `data1` et dans le même ordre, le numéro du cluster auquel il est affecté. Deux points sont dans le même cluster si et seulement s'ils ont le même numéro :

- `[0 1 2 2 2]` : 1 seul dans le cluster 0, 2 seul dans le cluster 1, et 18, 20, 31 ensemble dans le cluster 2. Ce sont les clusters {1}, {2}, {18, 20, 31} obtenus avec les centroïdes initiaux `[[1], [2], [18]]` (SSE 98.0) ;
- `[0 0 1 1 2]` : {1, 2}, {18, 20} et {31}, les clusters obtenus avec `[[18], [20], [31]]` (SSE 2.5).

Les numéros eux-mêmes n'ont pas d'importance : seul compte quels points partagent le même numéro. Ici, le cluster j de SciKit-Learn part du j-ième centroïde initial, comme dans notre code, donc les numéros coïncident aussi.

Comparaison sur les Exercices 1 à 3. Pour chaque configuration, nous lançons notre fonction (avec `retourner_clusters=True`) et celle de SciKit-Learn, comme dans l'énoncé :

```
KMeans(n_clusters=k, n_init=1, init=array(centroides_initiaux)).fit(data)
```

Avec `n_init=1`, SciKit-Learn fait une seule exécution à partir des centroïdes donnés, comme notre fonction. Ensuite :

- `labels_vers_clusters` transforme `labels_` en une liste de clusters, au même format que notre code ;
- `memes_clusters` compare les deux partitions sans tenir compte de la numérotation des clusters (chaque clustering est mis sous une forme triée avant la comparaison). Comparer directement les numéros serait trop strict : deux implémentations peuvent trouver les mêmes clusters avec des numéros différents ;
- la SSE de notre code est comparée à `inertia_`, qui est la même quantité (somme des carrés des distances de chaque point à son centroïde). Nous utilisons `math.isclose` et non `==`, car les deux codes ne font pas le calcul en virgule flottante de la même façon : pour `[[1], [2], [18]]`, SciKit-Learn donne `98.00000000000003` et notre code `98.0`.

Exercice	Centroïdes initiaux	SSE	Mêmes clusters	Même SSE
1	`[[1], [2], [18]]`	98.0	OUI	OUI
1	`[[18], [20], [31]]`	2.5	OUI	OUI
2 (a)	`[[2], [6]]`	74.0	OUI	OUI
2 (b)	`[[12], [24]]`	74.0	OUI	OUI
3	M1, M2	15.5	OUI	OUI
3	M4, M6	10.67	OUI	OUI

SciKit-Learn trouve exactement les mêmes clusters et la même SSE que notre code dans les six cas, y compris les deux minima locaux (98.0 et 15.5). Avec les mêmes centroïdes initiaux, SciKit-Learn s'arrête donc dans les mêmes minima locaux : ils viennent de l'algorithme k-means lui-même, pas de notre implémentation.

2 Clustering hiérarchique

2.1 Matrice de distance

Le code de cette section est dans `section2_hierarchique.py`. La commande suivante génère les deux figures de la section et affiche les fusions successives :

```
.venv/bin/python section2_hierarchique.py
```

Format des données. Comme indiqué dans l'énoncé, la matrice de distance est donnée à `hierarchy.linkage` au format compact 1D : on lit les valeurs au-dessus de la diagonale, ligne par ligne, c'est-à-dire $d(p_1,p_2), d(p_1,p_3), d(p_1,p_4), d(p_1,p_5), d(p_2,p_3), \ldots, d(p_4,p_5)$. Pour 5 points, cela fait $5 \times 4/2 = 10$ valeurs. La diagonale (toujours 0) et la partie sous la diagonale (identique, par symétrie) n'apportent aucune information.

```
data = [0.1, 0.9, 0.35, 0.8, 0.3, 0.4, 0.5, 0.6, 0.7, 0.2]
```

Fonctions. Le script définit deux fonctions :

```
dendrogramme(donnees, labels, methode, ax, orientation="top")
figure_single_complete(donnees, labels, titre, fichier)
```

— `dendrogramme` lance le clustering hiérarchique avec `hierarchy.linkage(donnees, method=methode)`, où `methode` vaut `"single"` ou `"complete"`, puis dessine le dendrogramme avec `hierarchy.dendrogram` sur les axes `ax`. `labels` donne le nom des feuilles (`p1` à `p5`). Elle retourne la matrice de linkage `Z`. `donnees` peut être une matrice de distance au format compact ou une liste de points, puisque `linkage` accepte les deux formats : c'est cette fonction que nous réutilisons dans l'étude de cas, avec `orientation="right"`.

— `figure_single_complete` place les dendrogrammes single link et complete link côte à côte, comme dans l'énoncé, et sauvegarde la figure dans `figures/<fichier>`. Elle affiche aussi chaque ligne de `Z` : les deux clusters fusionnés, la distance de fusion et la taille du nouveau cluster. Les points p_1 à p_5 portent les numéros 0 à 4, et le cluster créé à la i-ème fusion porte le numéro $4 + i$.

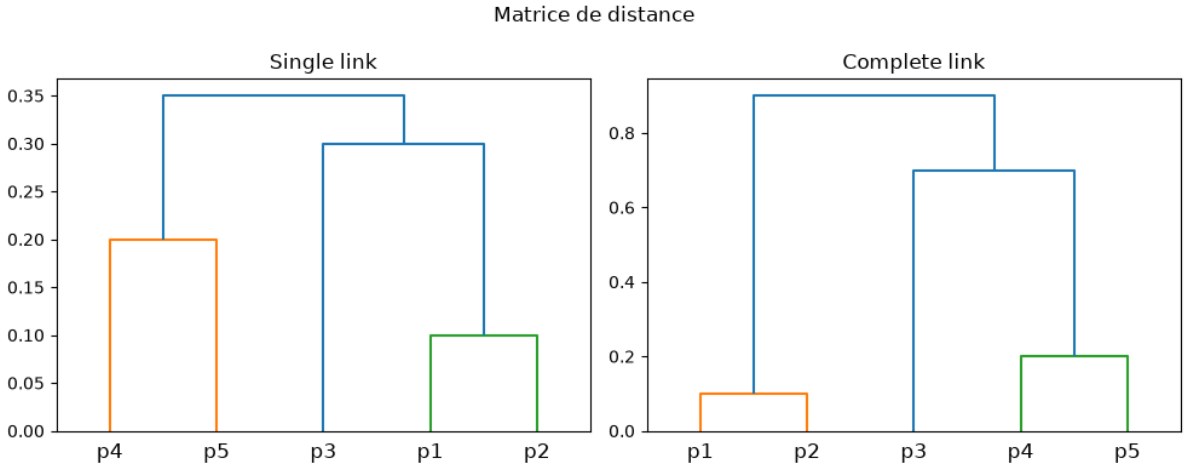


FIGURE 3 – Dendrogrammes obtenus avec la matrice de distance (fichier `figures/s2_distance.png`).

Résultat. Les dendrogrammes de la Figure 3 sont les mêmes que ceux de l'énoncé : même ordre des feuilles (p4, p5, p3, p1, p2 en single link et p1, p2, p3, p4, p5 en complete link) et mêmes hauteurs de fusion. Les fusions affichées par le script permettent de les vérifier à la main :

Étape	Single link (min)	Complete link (max)
1	$\{p_1, p_2\}$ à 0.10	$\{p_1, p_2\}$ à 0.10
2	$\{p_4, p_5\}$ à 0.20	$\{p_4, p_5\}$ à 0.20
3	p_3 rejoint $\{p_1, p_2\}$ à 0.30	p_3 rejoint $\{p_4, p_5\}$ à 0.70
4	tout fusionne à 0.35	tout fusionne à 0.90

Les deux premières fusions sont les mêmes, car entre deux points isolés, single link et complete link donnent la même distance. La différence apparaît pour p_3 :

— en single link, la distance entre deux clusters est la plus petite distance entre leurs points. $d(p_3, \{p_1, p_2\}) = \min(0.9, 0.3) = 0.3$ est plus petite que $d(p_3, \{p_4, p_5\}) = \min(0.6, 0.7) = 0.6$, donc p_3 rejoint $\{p_1, p_2\}$;

— en complete link, c'est la plus grande distance. $d(p_3, \{p_4, p_5\}) = \max(0.6, 0.7) = 0.7$ est plus petite que $d(p_3, \{p_1, p_2\}) = \max(0.9, 0.3) = 0.9$, donc p_3 rejoint $\{p_4, p_5\}$.

La dernière fusion se fait à 0.35 en single link (la plus petite distance entre les deux groupes, $d(p_1, p_4)$) et à 0.9 en complete link (la plus grande, $d(p_1, p_3)$).

2.2 Matrice de similarité

Conversion en distances. `linkage` attend des distances : deux points proches doivent avoir une petite valeur. Une similarité fait l'inverse (1 pour deux points identiques), donc nous la convertissons d'abord avec distance$(p_i, p_j) = 1 -$ similarité(p_i, p_j), comme demandé dans l'énoncé :

```
data_sim = squareform(similarite_vers_distance(sim))
```

— `sim` est la matrice 5×5 de l'énoncé, recopiée telle quelle, et `similarite_vers_distance` calcule `1 - sim` sur toute la matrice ;
— `squareform` (de `scipy.spatial.distance`) transforme la matrice carrée obtenue en format compact 1D, comme à la section précédente. Nous préférons cette fonction à une conversion à la main, car elle vérifie que la matrice est symétrique et que sa diagonale est nulle : une erreur de recopie de la matrice serait détectée.

Le script affiche les distances obtenues :

```
[0.9, 0.59, 0.45, 0.65, 0.36, 0.53, 0.02, 0.56, 0.15, 0.24]
```

Ensuite, nous appelons la même fonction `figure_single_complete` qu'avec la matrice de distance : seul le jeu de données change.

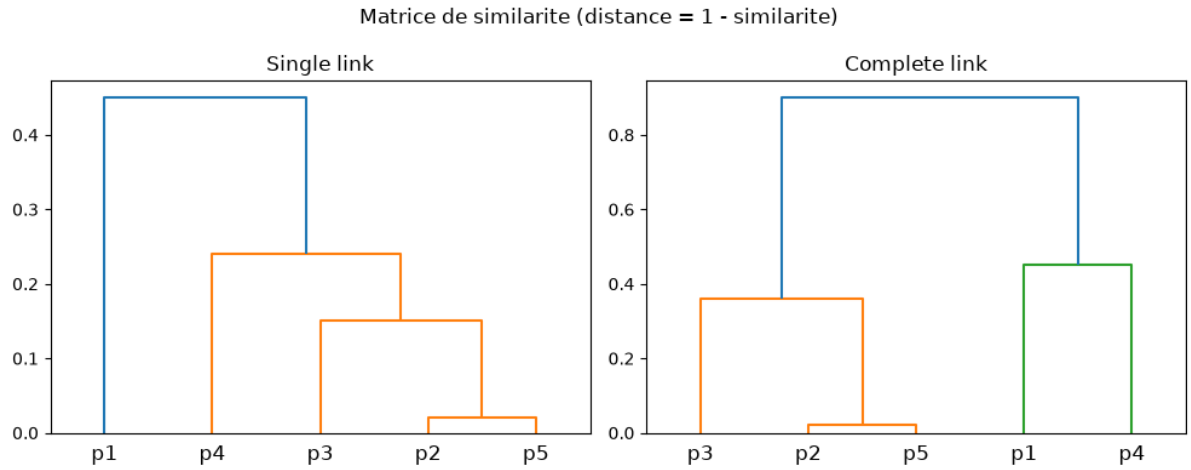


FIGURE 4 – Dendrogrammes obtenus avec la matrice de similarité convertie en distances (fichier `figures/s2_similarite.png`).

Résultat. Les dendrogrammes de la Figure 4 sont les mêmes que ceux de l'énoncé : même ordre des feuilles (p1, p4, p3, p2, p5 en single link et p3, p2, p5, p1, p4 en complete link) et mêmes hauteurs de fusion. Seules certaines couleurs de branches diffèrent, ce qui ne change pas le clustering.

Étape	Single link (min)	Complete link (max)
1	$\{p_2, p_5\}$ à 0.02	$\{p_2, p_5\}$ à 0.02
2	p_3 rejoint $\{p_2, p_5\}$ à 0.15	p_3 rejoint $\{p_2, p_5\}$ à 0.36
3	p_4 rejoint $\{p_2, p_3, p_5\}$ à 0.24	$\{p_1, p_4\}$ à 0.45
4	p_1 rejoint le reste à 0.45	tout fusionne à 0.90

En single link, les points rejoignent un par un le même cluster (p_3, puis p_4, puis p_1). C'est l'effet de chaîne du single link : il suffit qu'un point soit proche d'un seul point du cluster. Par exemple, p_4 rejoint $\{p_2, p_3, p_5\}$ à 0.24 grâce à $d(p_4, p_5) = 0.24$, alors qu'il est à 0.53 de p_2 et à 0.56 de p_3. En complete link, p_4 est jugé sur sa plus grande distance au cluster (0.56), donc il préfère

s'associer à p_1 (0.45). On obtient deux groupes plus compacts, $\{p_2, p_3, p_5\}$ et $\{p_1, p_4\}$, qui ne fusionnent qu'à la fin, à 0.9 ($d(p_1, p_2)$).

3 Étude de cas

3.1 Exercice 1 : préparation des données

Le code de l'étude de cas est dans `section3_etude_de_cas.py`. La commande suivante fait les Exercices 1 et 2, affiche les résultats et génère les figures `figures/s3_*.png` :

```
.venv/bin/python section3_etude_de_cas.py
```

Lecture du fichier. La fonction `lire_donnees` lit `data/Country-data.csv` avec le module `csv` de Python, saute la ligne d'en-tête et retourne deux listes :

— `countries` : les 167 noms de pays (première colonne), qui servent à étiqueter les figures ;
— `data_country` : 167 listes de 9 nombres (les 9 autres colonnes, converties en `float`).

Nous utilisons le module `csv` plutôt qu'un simple `split(",")`, car quatre noms de pays contiennent une virgule et sont donc entre guillemets dans le fichier (`"Congo, Dem. Rep."`, `"Congo, Rep."`, `"Macedonia, FYR"` et `"Micronesia, Fed. Sts."`). Un `split` couperait ces lignes en 11 morceaux au lieu de 10. Le script affiche le début de `data_country`, qui est bien celui de l'énoncé :

```
[[90.2, 10.0, 7.58, 44.9, 1610.0, 9.44, 56.2, 5.82, 553.0],
 [16.6, 28.0, 6.55, 48.6, 9930.0, 4.49, 76.3, 1.65, 4090.0], ...
```

Pourquoi une ACP sur données centrées réduites. L'énoncé donne deux raisons :

— avec 9 variables, on ne peut pas visualiser les points et les clusters ; l'ACP projette les données sur les 2 directions qui gardent le plus de variance, ce qui permet de les dessiner en 2D ;
— les variables n'ont pas la même échelle (`gdpp` et `income` se comptent en milliers, `total_fer` en unités). Sans réduction, ces variables domineraient les distances. Après centrage-réduction, chaque variable a une moyenne de 0 et un écart type de 1, donc le même poids.

Squelette complété. La fonction `preparer_donnees` reprend le squelette de l'énoncé ligne par ligne (calcul de `X`, `n`, `p`, `Y`, `Z` et `R`). Comme `Z` est centrée réduite, $\texttt{R} = \frac{1}{n} Z^\top Z$ est la matrice des corrélations entre les 9 variables. Nous avons dû compléter la ligne `eigenvectors = eig(R)[1]`, pour trois raisons (extrait du script, commentaires abrégés) :

```
valeurs_propres, eigenvectors = eig(R)
# 1) R est symetrique reelle : ses valeurs propres sont reelles
assert np.allclose(valeurs_propres.imag, 0) and np.allclose(eigenvectors.imag, 0)
valeurs_propres, eigenvectors = valeurs_propres.real, eigenvectors.real
# 2) eig ne trie pas les valeurs propres : ordre decroissant
ordre = argsort(valeurs_propres)[::-1]
valeurs_propres, eigenvectors = valeurs_propres[ordre], eigenvectors[:, ordre]
# 3) le signe d'un vecteur propre est arbitraire : on prend celui de l'enonce
for c in range(2):
    if matmul(Z[0], eigenvectors[:, c]) * REFERENCE_ENONCE[c] < 0:
        eigenvectors[:, c] = -eigenvectors[:, c]
```

1. Sur notre machine, `eig` renvoie des nombres complexes (type `complex128`), dont la partie imaginaire est nulle. Le reste du code a besoin de nombres réels. Comme R est symétrique réelle, ses valeurs propres sont forcément réelles : nous vérifions que la partie imaginaire est nulle (`assert`), puis nous gardons la partie réelle.
2. `eig` ne range pas les valeurs propres dans un ordre particulier. Sur notre machine, les deux premières sont bien les plus grandes, mais les trois dernières sont dans le désordre (0.067, 0.088, 0.113). Comme `eigenvectors[:, 0]` et `eigenvectors[:, 1]` doivent être les deux composantes principales, nous trions les valeurs propres par ordre décroissant (avec les vecteurs propres associés) pour ne pas dépendre de cet ordre.
3. Si v est un vecteur propre, $-v$ en est un aussi, et `eig` peut renvoyer l'un ou l'autre selon la machine ou la version de NumPy. Changer le signe d'une composante revient à faire une symétrie de la figure : les distances entre pays, donc les clusterings, ne changent pas. Nous choisissons le signe qui donne le même premier point que l'énoncé (`REFERENCE_ENONCE = [2.91, -0.1]`), pour que nos figures soient orientées comme les siennes.

Résultat. Le script affiche le début de `data_reduced`, arrondi à deux décimales comme dans l'énoncé, et c'est bien celui de l'énoncé :

```
data_reduced commence par [[2.91, -0.1], [-0.43, 0.59]]
```

Il affiche aussi la part de variance expliquée par chaque composante, égale à λ/p. En effet, la diagonale de la matrice des corrélations ne contient que des 1, donc la somme des 9 valeurs propres (la trace de R) vaut $p = 9$. La première composante explique 46.0 % de la variance et la seconde 17.2 %, soit 63.1 % au total. La projection en 2D garde donc environ deux tiers de l'information : deux pays proches sur nos figures ne sont pas forcément proches sur les 9 variables, ce qu'il faudra garder en tête pour interpréter les clusters.

3.2 Exercice 2 : clustering et interprétation

Clustering k-means

Méthode. Comme demandé, nous réutilisons la fonction `kmeans` de la Section 1 (importée depuis `section1_kmeans.py`), appliquée à `data_reduced` avec $k = 2$ puis $k = 3$ et des centroïdes initiaux tirés au hasard (`centroides_initiaux=None`). Deux détails :

— avec 167 points, l'affichage de toutes les itérations serait illisible : le script le masque et n'affiche qu'un résumé (nombre d'itérations, SSE, taille des clusters). Nous récupérons les clusters finaux avec `retourner_clusters=True` pour dessiner la figure ;

— pour que le résultat soit reproductible, nous fixons la graine du tirage aléatoire avant chaque appel (`random.seed`). Nous avons choisi les graines 0 pour $k = 2$ et 13 pour $k = 3$ parce qu'elles redonnent les clusterings des Figures 1 et 2 de l'énoncé : en comparant les figures, en particulier les pays à la frontière entre deux clusters, nous retrouvons les mêmes clusters et les mêmes couleurs (seule la couleur de Gabon, pour $k = 3$, n'est pas lisible avec certitude sur la figure de l'énoncé).

La fonction `figure_kmeans` dessine chaque cluster avec sa couleur (la même liste de couleurs qu'en Section 1), son centroïde avec un $\times$, et annote chaque point avec le nom du pays.

k	Graine	Itérations	SSE	Taille des clusters	Figure
2	0	14	503.12	89 (rouge), 78 (bleu)	5
3	13	8	373.47	41 (rouge), 48 (bleu), 78 (orange)	6

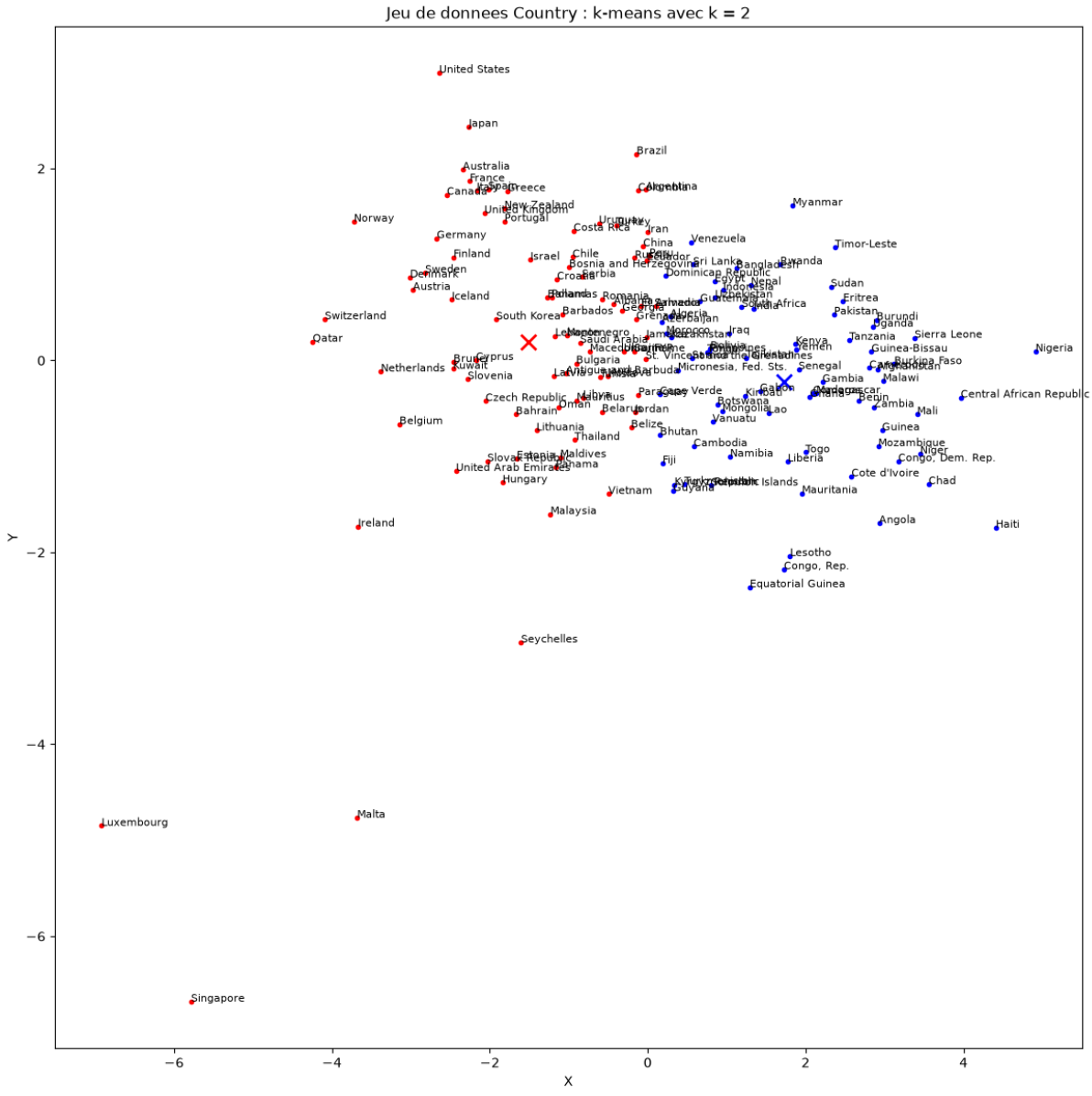


FIGURE 5 – k-means avec $k = 2$ sur `data_reduced` (fichier `figures/s3_kmeans_k2.png`).

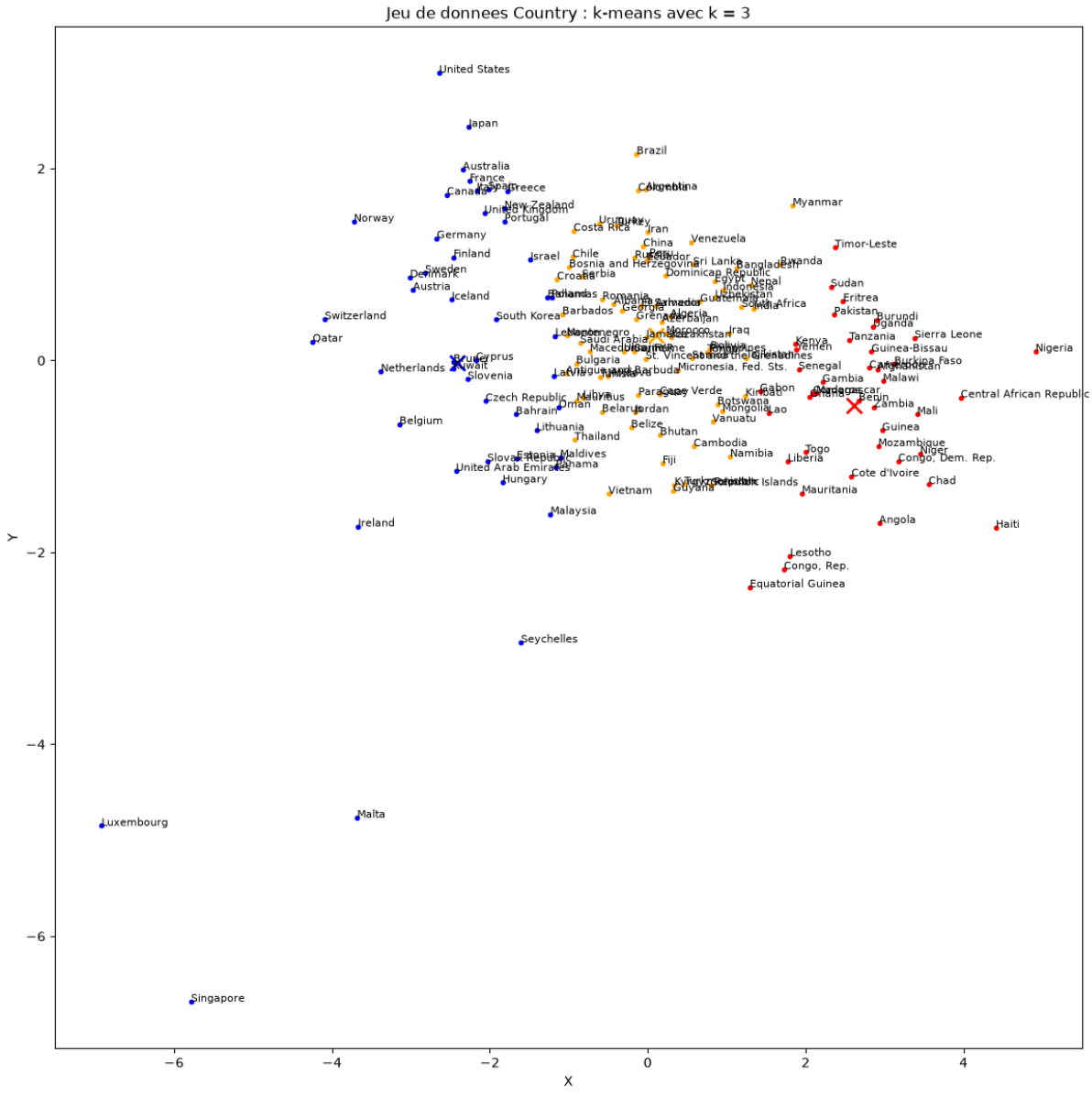


FIGURE 6 – k-means avec $k = 3$ sur `data_reduced` (fichier `figures/s3_kmeans_k3.png`).

Aurait-on pu obtenir des clusterings très différents ? Pour répondre à cette question de l'énoncé, le script relance k-means avec 200 graines différentes (0 à 199) et regroupe les clusterings identiques, quelle que soit la numérotation des clusters. Il affiche les clusterings les plus fréquents :

Tirages	SSE	Taille des clusters	Remarque
$k = 2$: 3 clusterings différents			
107	503.12	78, 89	clustering de la Figure 5
85	503.02	76, 91	Azerbaijan et Dominican Republic changent de cluster
8	762.70	4, 163	{Ireland, Luxembourg, Malta, Singapore} isolés
$k = 3$: 30 clusterings différents, SSE entre 354.52 et 415.12 (les 6 plus fréquents)			
56	373.47	41, 48, 78	clustering de la Figure 6
27	374.82	23, 70, 74	
24	370.88	40, 47, 80	
17	371.03	41, 45, 81	
9	370.96	41, 47, 79	
9	354.53	3, 72, 92	{Luxembourg, Malta, Singapore} isolés

— Pour $k = 2$, le résultat est stable : 192 tirages sur 200 donnent le clustering de la Figure 5 ou un clustering presque identique, où seuls deux pays changent de cluster. Les 8 autres tirages donnent un clustering très différent et nettement moins bon (SSE 762.70), qui isole quatre petits pays : d'après le fichier de données, ce sont les quatre pays dont les exportations sont les plus élevées (de 103 à 200 % du PIB, contre une médiane de 35 %).
— Pour $k = 3$, le résultat dépend beaucoup plus du tirage : 30 clusterings différents, et celui de la Figure 6 n'est obtenu que dans 56 tirages sur 200 (28 %). Beaucoup de variantes ont des tailles proches (40 à 41, 45 à 48, 78 à 81 pays), mais d'autres sont nettement différentes, par exemple 23, 70 et 74 pays.
— La SSE la plus basse ne donne pas forcément le clustering le plus utile. Pour $k = 3$, le clustering qui isole Luxembourg, Malta et Singapore a une SSE plus faible (354.53) que celui de l'énoncé (373.47), mais l'un de ses clusters ne contient que 3 pays, à l'écart des autres sur la figure.

Donc oui, avec des centroïdes initiaux tirés au hasard, nous aurions pu obtenir des clusterings considérablement différents de ceux de l'énoncé, surtout pour $k = 3$.

Clustering hiérarchique

Méthode. Comme demandé, nous réutilisons la fonction `dendrogramme` de la Section 2 (importée depuis `section2_hierarchique.py`), avec les méthodes single link et complete link. La subtilité signalée par l'énoncé est que nous avons maintenant une liste de points (`data_reduced`) et non une matrice de distance. Nous n'avons rien à changer : `hierarchy.linkage` reconnaît une liste de points et calcule lui-même les distances euclidiennes entre eux, c'est-à-dire la même distance que pour k-means. Pour que les 167 noms de pays restent lisibles, le dendrogramme est dessiné à l'horizontale (`orientation="right"`) dans une figure haute, comme les Figures 3 et 4 de l'énoncé.

Comparaison avec l'énoncé. Avec 167 feuilles, une comparaison à l'œil ne suffit pas. Nous avons donc extrait le texte des Figures 3 et 4 du PDF de l'énoncé avec `pdftotext`, qui donne la position de chaque mot, pour obtenir l'ordre des 167 pays de haut en bas. Nous l'avons comparé

à l'ordre des feuilles de nos dendrogrammes : il est identique, en single link comme en complete link. Les graduations de l'axe des distances sont aussi les mêmes (de 0 à 2.5 en single link, de 0 à 12 en complete link). Cette vérification a été faite à part et ne fait pas partie des scripts rendus.

Clusters obtenus en coupant les dendrogrammes. Pour comparer avec k-means, le script coupe chaque dendrogramme en 2 puis en 3 clusters (`hierarchy.fcluster` avec le critère `maxclust`, ce qui revient à couper l'arbre horizontalement à la hauteur qui donne ce nombre de clusters) et affiche la taille des clusters :

Méthode	Clusters	Tailles	Petits clusters (5 pays au plus)
Single link	2	2, 165	{Luxembourg, Singapore}
Single link	3	2, 164, 1	{Luxembourg, Singapore}, {Malta}
Complete link	2	3, 164	{Luxembourg, Malta, Singapore}
Complete link	3	3, 41, 123	{Luxembourg, Malta, Singapore}

Dans les deux cas, les premiers clusters à se séparer du reste sont les quelques pays isolés en bas des Figures 5 et 6. En single link, presque tous les autres pays forment un seul grand cluster : c'est l'effet de chaîne déjà observé en Section 2. En complete link, la coupe en 3 clusters sépare en plus un groupe de 41 pays des 123 autres.

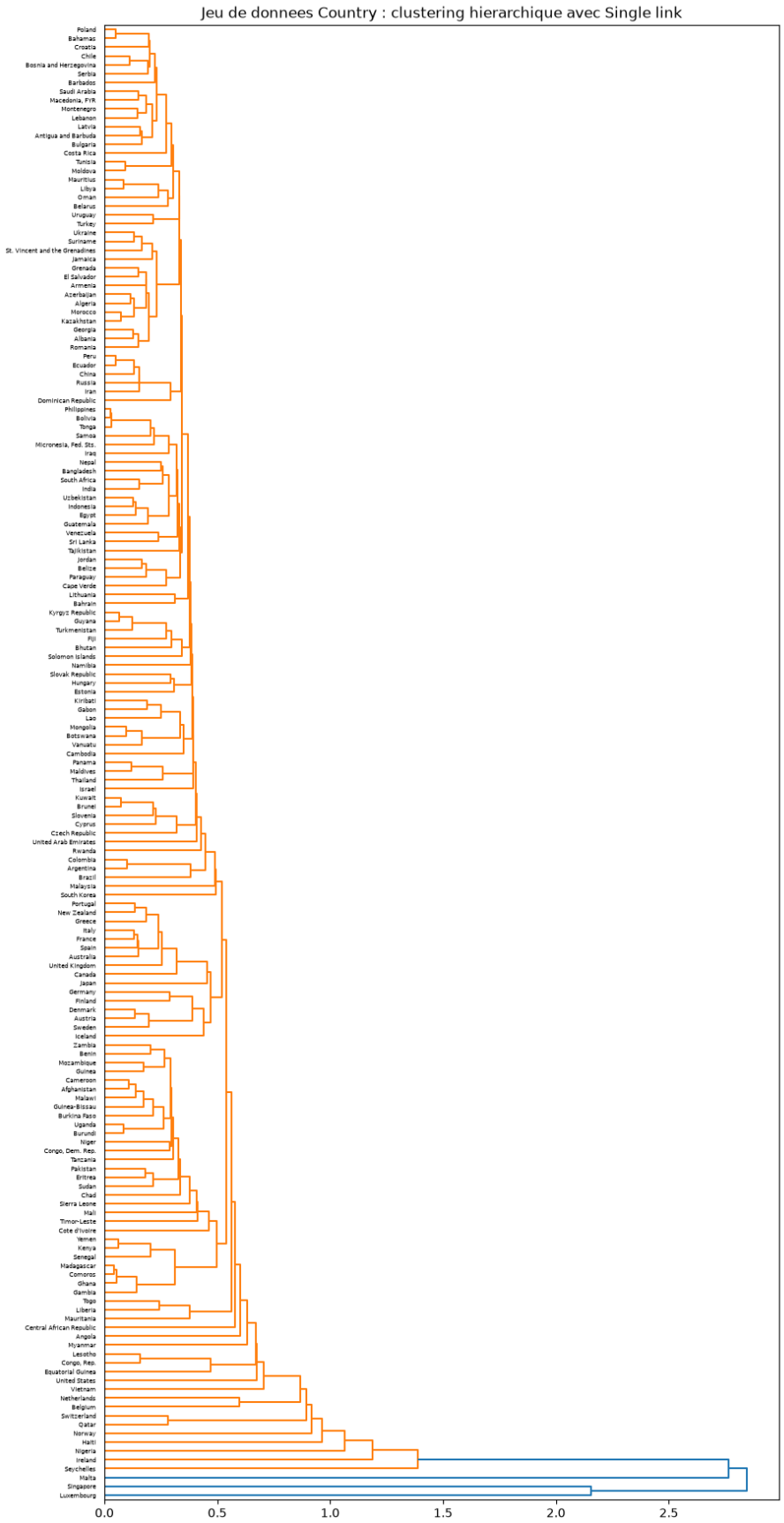


FIGURE 7 – Clustering hiérarchique single link sur `data_reduced` (fichier `figures/s3_single.png`).

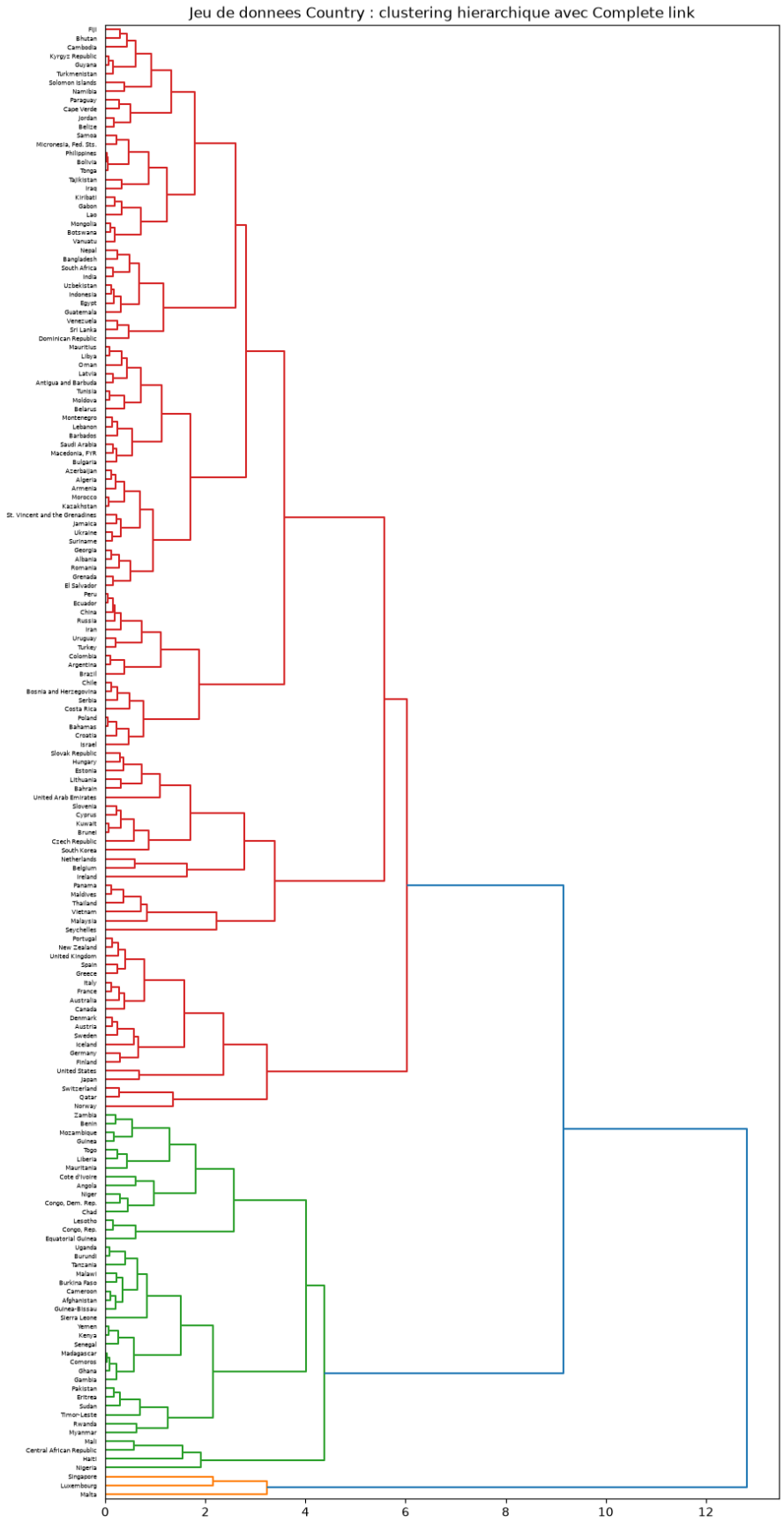


FIGURE 8 – Clustering hiérarchique complete link sur `data_reduced` (fichier `figures/s3_complete.png`).

Interprétation

Ce que représentent les axes. Pour interpréter les figures, il faut d'abord savoir ce que mesurent les deux composantes principales. Le script affiche la corrélation de chaque variable initiale avec X et avec Y ; voici les plus fortes :

Axe	Corrélation positive	Corrélation négative
X (46.0 %)	child_mort (+0.85), total_fer (+0.82)	life_expec (−0.87), income (−0.81), gdpp (−0.80), exports (−0.58)
Y (17.2 %)	—	imports (−0.84), exports (−0.76)

L'axe X mesure donc le niveau de développement : plus un pays est à droite, plus la mortalité infantile et la fécondité sont élevées, et plus le revenu, le PIB par habitant et l'espérance de vie sont faibles. L'axe Y mesure surtout le poids du commerce extérieur : plus un pays est bas, plus ses exportations et importations (en % du PIB) sont élevées. Les pays sont plus étalés selon X que selon Y, donc c'est surtout le niveau de développement qui décide des distances, et donc des clusters.

Équilibre et sens des clusters k-means. Le tableau suivant donne la moyenne de quelques variables initiales dans chaque cluster, affichée par le script (mortalité des enfants de moins de 5 ans pour 1000 naissances, revenu et PIB par habitant, espérance de vie en années, nombre d'enfants par femme, exportations en % du PIB) :

Clustering	Cluster (pays)	child_mort	income	gdpp	life_expec	total_fer	exports
$k = 2$	rouge (89)	10.4	27 691	22 216	77.0	1.9	49.9
	bleu (78)	70.1	5 111	2 408	63.2	4.2	31.1
$k = 3$	rouge (41)	98.6	3 311	1 585	59.0	5.2	28.9
	orange (78)	26.2	10 578	5 326	71.4	2.5	35.7
	bleu (48)	6.3	39 631	35 096	79.0	1.8	60.3

— Avec $k = 2$, les clusters sont équilibrés (89 et 78 pays) et ont un sens : d'un côté les pays développés ou intermédiaires, de l'autre les pays en développement. Mais le cluster rouge est très hétérogène : il contient des pays aussi différents que Norway (PIB par habitant de 87 800) et Vietnam (1 310).

— Avec $k = 3$, les clusters restent assez équilibrés (41, 48 et 78 pays) et les moyennes sont bien ordonnées : pays pauvres (rouge), intermédiaires (orange) et riches (bleu). C'est le clustering que nous trouvons le plus parlant. Le cluster bleu a aussi les exportations les plus élevées (60.3 % du PIB en moyenne), à cause de l'axe Y : nous y revenons plus bas.

Proximité dans les clusters et sur la carte du monde. Pour $k = 3$, les clusters ressemblent à de grandes régions du monde :

— rouge : 35 des 41 pays sont en Afrique ; les 6 autres sont Afghanistan, Haiti, Lao, Pakistan, Timor-Leste et Yemen ;

— bleu : Europe de l'Ouest et du Nord, pays baltes et une partie de l'Europe centrale (Czech Republic, Hungary, Poland, Slovak Republic, Slovenia), United States, Canada, Japan, South Korea, Australia, New Zealand, ainsi que les pays du Golfe sauf Saudi Arabia (Bahrain, Kuwait, Oman, Qatar, United Arab Emirates) ;

— orange : presque toute l'Amérique latine, la plupart des pays des Balkans et de l'ex-URSS, l'Afrique du Nord (Algeria, Egypt, Libya, Morocco, Tunisia), l'Afrique australe (Botswana, Namibia, South Africa) et une grande partie de l'Asie (China, India, Indonesia, Vietnam, ...).

La géographie n'est pas utilisée par le clustering : elle apparaît seulement parce que des pays voisins ont souvent un niveau de développement proche. Plusieurs proximités sont celles que nous attendions : sur la Figure 6, les pays nordiques (Norway, Sweden, Denmark, Finland) sont côte à côte, de même que France, Italy et Spain, ou que les pays du Sahel (Mali, Niger, Chad). D'autres nous ont surpris :

- **Haiti et Dominican Republic**, qui se partagent la même île, sont dans des clusters différents : Haiti est dans le cluster rouge, à côté de Chad, Niger et Congo, Dem. Rep., alors que Dominican Republic est dans le cluster orange. Dans le fichier, Haiti a la mortalité infantile la plus élevée (208) et l'espérance de vie la plus faible (32.1 ans) des 167 pays.
- **Equatorial Guinea et Gabon** sont dans le cluster des pays pauvres alors que leur revenu par habitant est élevé (33 700 pour Equatorial Guinea, 29e sur 167 ; 15 400 pour Gabon). Leur mortalité infantile (111 et 63.7) et leur espérance de vie (60.9 et 62.9 ans) sont celles des pays pauvres. Or trois des cinq variables les plus liées à X concernent la santé et la démographie : un revenu élevé ne suffit pas à placer un pays à gauche.
- **Malaysia, Maldives, Panama et Seychelles** sont dans le cluster des pays riches, avec un PIB par habitant de 7 100 à 10 800, alors que Saudi Arabia (19 300) est dans le cluster orange. Ces quatre pays ont un commerce extérieur très élevé (exportations de 70 à 94 % du PIB) : ils sont bas sur la figure, donc plus loin du centroïde orange que du bleu. Saudi Arabia, au contraire, est au milieu du nuage, près du centroïde orange. Le cluster bleu regroupe donc les pays riches, mais aussi des pays moins riches très ouverts au commerce.
- **Luxembourg, Malta et Singapore** sont très éloignés sur la carte, mais proches les uns des autres sur la figure et isolés de tous les autres pays. Ce sont, avec Ireland, les pays dont les exportations sont les plus élevées (de 103 à 200 % du PIB) : leur proximité vient de leur économie, pas de leur position géographique.

Clusterings hiérarchiques. Les coupes des dendrogrammes donnent des clusters très déséquilibrés :

- en single link (2, 165 pays puis 2, 164, 1 pays), le clustering n'a pas de sens pour regrouper les pays. Dans le grand nuage, chaque pays a un voisin proche, donc les pays sont reliés de proche en proche en un seul cluster (effet de chaîne) et seuls les pays isolés se séparent. Le single link est utile pour repérer ces pays atypiques, pas pour former des groupes ;
- en complete link, la coupe en 3 clusters (3, 41 et 123 pays) a plus de sens. Le script la compare au clustering k-means de la Figure 6 : le cluster de 41 pays a 39 pays en commun avec le cluster rouge de k-means. Seuls Myanmar et Rwanda (en plus) et Gabon et Lao (en moins) diffèrent, quatre pays proches de la frontière entre les clusters rouge et orange. Deux méthodes différentes trouvent donc presque le même groupe de pays les plus pauvres, ce qui nous rend plus confiants dans ce groupe. En revanche, le cluster de 123 pays mélange les pays riches et intermédiaires (45 pays du cluster bleu et 76 du cluster orange), et Luxembourg, Malta et Singapore forment un cluster à eux seuls : comme ils sont loin de tous les autres pays, les ajouter à un cluster augmenterait beaucoup sa distance maximale, donc le complete link ne les fusionne qu'à la fin.

Conclusion. Sur ce jeu de données, k-means avec $k = 3$ donne le clustering le plus utile : des clusters assez équilibrés, qui correspondent à trois niveaux de développement et ressemblent à de grandes régions du monde. Le complete link confirme le groupe des pays les plus pauvres, et le single link surtout les pays atypiques. Il faut garder deux limites en tête : les deux composantes ne conservent que 63.1 % de la variance, et l'axe Y donne beaucoup de poids au commerce extérieur, ce qui rapproche des pays très ouverts au commerce quel que soit leur niveau de développement ou leur position sur la carte.